Finite State Machines
in VHDL

A Complete Hands-On Guide
From Beginner to Advanced

```vhdl
-- The skeleton you will master in this guide
architecture myfsm of fsm is
    type state_type is (S0, S1, S2, S3);
    signal state, nextState : state_type;
begin
    -- Process 1 : State Register
    process (reset, clk) ...

    -- Process 2 : Next-State + Output Logic
    process (state, X) ...
end myfsm;
```

Subject	Digital System Design (DSD)
Tool	Intel Quartus Prime / ModelSim
VHDL Std	IEEE 1164
Levels	1 (Beginner) → 6 (Advanced)

This guide follows the 2-process Mealy/Moore template and the exact coding style used in your lab sessions.

Contents

1 FSM Theory — The Foundation **2**
1.1 What is a Finite State Machine? . . . 2
1.2 Moore vs Mealy — Key Difference . . . 2
1.3 The 2-Process Template (Your Style) . . . 2

2 Level 1 — Your First FSM (Toggle Machine) **3**
2.1 Problem . . . 3
2.2 State Diagram . . . 3
2.3 State Table . . . 3
2.4 VHDL Implementation . . . 3

3 Level 2 — Sequence Detector (Moore) **6**
3.1 Problem . . . 6
3.2 State Diagram . . . 6
3.3 State Meaning Table . . . 6
3.4 VHDL Implementation . . . 6

4 Level 3 — Mealy Machine **9**
4.1 Moore vs Mealy — Visual Comparison . . . 9
4.2 Problem — Same Sequence Detector, Mealy Style . . . 9
4.3 VHDL Implementation . . . 9

5 Level 4 — 3-Process FSM & Multiple Outputs **12**
5.1 Why a Third Process? . . . 12
5.2 Problem — Traffic Light Controller . . . 12

6 Level 5 — State Encoding Strategies **15**
6.1 Three Encoding Methods . . . 15
6.2 Forcing Encoding in VHDL . . . 15

7 Level 6 — FSM with Datapath (FSMD) **18**
7.1 What is an FSMD? . . . 18
7.2 Problem — Pulse-Width Counter . . . 18

8 Common Mistakes & How to Fix Them **21**
8.1 Bug Reference Table . . . 21
8.2 The Corrected Version of Your fsm.vhd . . . 22

9 Quick Reference — The Master Template **25**
9.1 FSM Design Checklist . . . 26
9.2 Level Summary Table . . . 27
9.3 The Golden Rules (Never Break These) . . . 27

1 FSM Theory — The Foundation

1.1 What is a Finite State Machine?

A **Finite State Machine (FSM)** is a sequential circuit that:

- Has a **finite** number of states.
- Is always in **exactly one** state at any clock edge.
- Moves to a **next state** based on the current state and inputs.
- Produces **outputs** based on the state (and sometimes inputs).

Core Formula

$$\text{Next State} = f(\text{Current State},\ \text{Inputs})$$

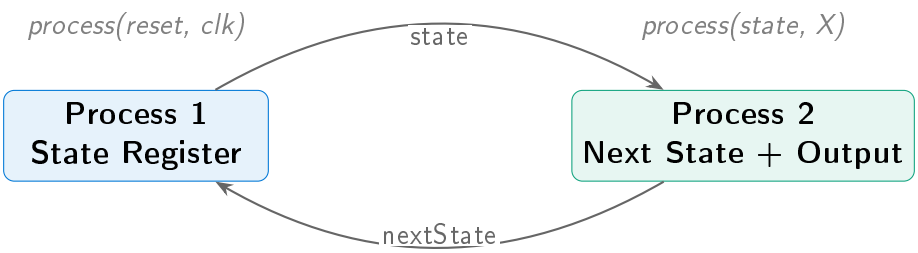
$$\text{Output} = g(\text{Current State}) \quad [\text{Moore}] \quad \text{or} \quad g(\text{Current State},\ \text{Inputs}) \quad [\text{Mealy}]$$

1.2 Moore vs Mealy — Key Difference

	Moore Machine	Mealy Machine
Output depends on	State only	State **and** Inputs
Output changes	Only at clock edge	Can change with input (async)
States needed	Usually more	Usually fewer
Diagram label	State bubble holds output	Arrow/transition holds output
VHDL output	Assigned in state register process or separate	Assigned inside combinational process

1.3 The 2-Process Template (Your Style)

Every FSM you write in this guide uses **exactly two processes**:



- **Process 1** — clocked, sensitive to `reset` and `clk`. Contains the flip-flops. Simple; never changes.
- **Process 2** — combinational, sensitive to `state` and all **inputs**. Contains the *brain* of your FSM.

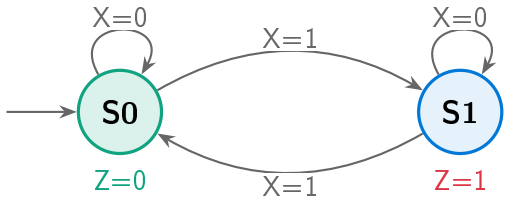
2 Level 1 — Your First FSM (Toggle Machine)

● Level 1 · Beginner — 2 States, 1 Input, 1 Output

2.1 Problem

Design a **Moore FSM** that toggles output `Z` every time input `X = 1`. The machine resets to state `S0` (`Z = 0`) when `reset = 1`.

2.2 State Diagram



Reading State Diagrams

- **Green circle** = Initial state (loaded on reset).
- **Arrow label** = input condition to take that transition.
- **Text inside bubble** = Moore output for that state.
- **Self-loop** = stay in same state when condition is met.

2.3 State Table

Current State	Input X	Next State	Output Z
S0	0	S0	0
S0	1	S1	0
S1	0	S1	1
S1	1	S0	1

2.4 VHDL Implementation

Level 1 — toggle_fsm.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity toggle_fsm is
    port (
        clk   : in  std_logic;   -- clock
        reset : in  std_logic;   -- async active-high reset
        X     : in  std_logic;   -- input
```

```
        Z      : out std_logic     -- output
    );
end toggle_fsm;

architecture myfsm of toggle_fsm is
    -- state encoding
    type state_type is (S0, S1);

    -- signals
    signal state, nextState : state_type;

begin

    -- ----------------------------------------
    -- Process 1 : State Register (clocked)
    -- ----------------------------------------
    process (reset, clk)
    begin
        if reset = '1' then
            state <= S0;                -- async reset to initial state
        elsif rising_edge(clk) then
            state <= nextState;         -- advance on clock edge
        end if;
    end process;

    -- ----------------------------------------
    -- Process 2 : Next-State + Output Logic
    -- ----------------------------------------
    process (state, X)
    begin
        -- default assignments (prevent latches)
        nextState <= state;
        Z         <= '0';

        case state is

            -- S0 : output Z = 0
            when S0 =>
                Z <= '0';
                if X = '1' then
                    nextState <= S1;
                else
                    nextState <= S0;
                end if;

            -- S1 : output Z = 1
            when S1 =>
                Z <= '1';
```

```
                    if X = '1' then
                        nextState <= S0;
                    else
                        nextState <= S1;
                    end if;

                -- safety net (always include this!)
                when others =>
                    nextState <= S0;
                    Z         <= '0';

            end case;
        end process;

end myfsm;
```

Key Observations from Level 1

1. `reset` is **in**, `clk` is **in** — never `out`.
2. `signal state, nextState : state_type;` — always specify the type.
3. Always write `when others` — it prevents undefined behavior on FPGA.
4. Default assignments at the top of Process 2 prevent **inferred latches**.

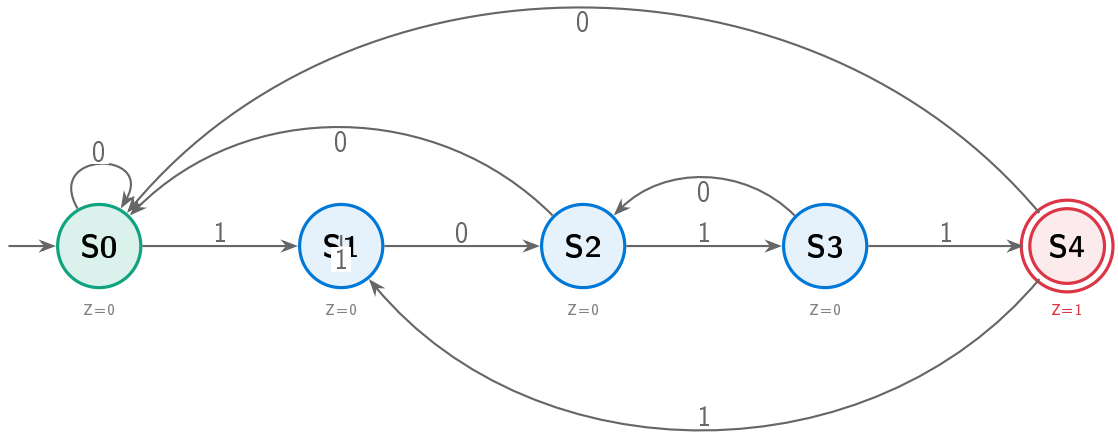
3 Level 2 — Sequence Detector (Moore)

❍ Level 2 · Easy-Intermediate — Multi-State Moore Machine

3.1 Problem

Detect the sequence **"1011"** in a serial input stream `X`. Output `Z = 1` for one clock cycle when the sequence is found. Use overlapping detection (the last bits of one match can be the start of the next).

3.2 State Diagram



3.3 State Meaning Table

State	Meaning	Bits seen so far
S0	Reset / no prefix matched	—
S1	Received 1	1
S2	Received 10	10
S3	Received 101	101
S4	Received 1011	1011 **(MATCH!)**

3.4 VHDL Implementation

Level 2 — seq_detector_1011.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity seq_detector_1011 is
    port (
        clk    : in  std_logic;
```

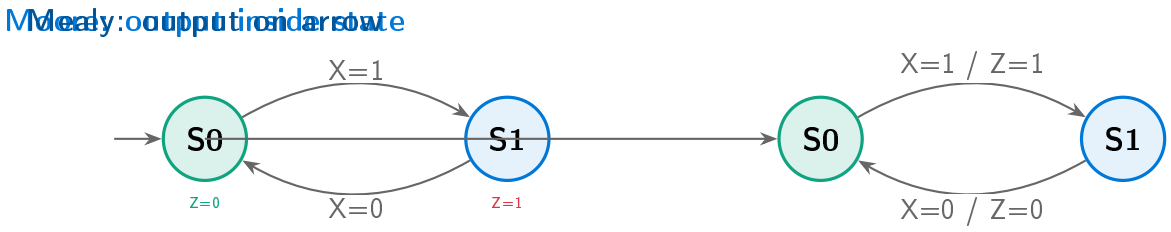
```vhdl
        reset : in  std_logic;
        X     : in  std_logic;
        Z     : out std_logic
    );
end seq_detector_1011;

architecture myfsm of seq_detector_1011 is
    -- state encoding
    type state_type is (S0, S1, S2, S3, S4);

    -- signals
    signal state, nextState : state_type;

begin

    -- ----------------------------------------
    -- Process 1 : State Register
    -- ----------------------------------------
    process (reset, clk)
    begin
        if reset = '1' then
            state <= S0;
        elsif rising_edge(clk) then
            state <= nextState;
        end if;
    end process;

    -- ----------------------------------------
    -- Process 2 : Next-State Logic
    -- ----------------------------------------
    process (state, X)
    begin
        -- defaults
        nextState <= S0;
        Z         <= '0';

        case state is

            -- S0 : no bits matched yet
            when S0 =>
                Z <= '0';
                if X = '1' then
                    nextState <= S1;    -- got '1'
                else
                    nextState <= S0;    -- got '0', stay
                end if;

            -- S1 : matched '1'
            when S1 =>
                Z <= '0';
```

```
                if X = '0' then
                    nextState <= S2;    -- got '10'
                else
                    nextState <= S1;    -- got '11', restart at '1'
                end if;

            -- S2 : matched '10'
            when S2 =>
                Z <= '0';
                if X = '1' then
                    nextState <= S3;    -- got '101'
                else
                    nextState <= S0;    -- got '100', reset
                end if;

            -- S3 : matched '101'
            when S3 =>
                Z <= '0';
                if X = '1' then
                    nextState <= S4;    -- got '1011'!
                else
                    nextState <= S2;    -- got '1010', overlap at '10'
                end if;

            -- S4 : full match '1011' found
            when S4 =>
                Z <= '1';               -- assert output for 1 cycle
                if X = '1' then
                    nextState <= S1;    -- '1' could start new match
                else
                    nextState <= S0;
                end if;

            when others =>
                nextState <= S0;
                Z         <= '0';

        end case;
    end process;

end myfsm;
```

4 Level 3 — Mealy Machine

❍ Level 3 · Intermediate — Mealy Output on Transitions

4.1 Moore vs Mealy — Visual Comparison



4.2 Problem — Same Sequence Detector, Mealy Style

Detect **"1011"** using a **Mealy** machine. Output `Z = 1` is asserted *on the transition* when the last bit is received, not in a dedicated output state. This saves one state compared to the Moore version.

4.3 VHDL Implementation

Level 3 — mealy_1011.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity mealy_1011 is
    port (
        clk   : in  std_logic;
        reset : in  std_logic;
        X     : in  std_logic;
        Z     : out std_logic
    );
end mealy_1011;

architecture myfsm of mealy_1011 is
    -- state encoding (one fewer state than Moore!)
    type state_type is (S0, S1, S2, S3);

    -- signals
    signal state, nextState : state_type;

begin

    -- ----------------------------------------
    -- Process 1 : State Register
    -- ----------------------------------------
    process (reset, clk)
```

```
    begin
        if reset = '1' then
            state <= S0;
        elsif rising_edge(clk) then
            state <= nextState;
        end if;
    end process;

    -- ----------------------------------------
    -- Process 2 : Next-State + Output (Mealy)
    -- Output Z is assigned on the TRANSITION
    -- ----------------------------------------
    process (state, X)
    begin
        -- defaults
        nextState <= S0;
        Z         <= '0';

        case state is

            -- S0 : no prefix
            when S0 =>
                if X = '1' then
                    nextState <= S1;
                    Z         <= '0';
                else
                    nextState <= S0;
                    Z         <= '0';
                end if;

            -- S1 : matched '1'
            when S1 =>
                if X = '0' then
                    nextState <= S2;
                    Z         <= '0';
                else
                    nextState <= S1;    -- '11' -> keep '1'
                    Z         <= '0';
                end if;

            -- S2 : matched '10'
            when S2 =>
                if X = '1' then
                    nextState <= S3;
                    Z         <= '0';
                else
                    nextState <= S0;
                    Z         <= '0';
                end if;

```

```
            -- S3 : matched '101'
            when S3 =>
                if X = '1' then
                    nextState <= S1;    -- overlap: last '1' starts new match
                    Z         <= '1';   -- OUTPUT HERE on the transition!
                else
                    nextState <= S2;    -- '1010' -> overlap at '10'
                    Z         <= '0';
                end if;

            when others =>
                nextState <= S0;
                Z         <= '0';

        end case;
    end process;

end myfsm;
```

Mealy Key Point

In Mealy, `Z` is assigned **inside the transition condition** (`if X = '1' then ... Z <= '1'`). It can change *between* clock edges whenever `X` changes. This is why Mealy needs one fewer state but can produce glitches on output.

5 Level 4 — 3-Process FSM & Multiple Outputs

❍ Level 4 · Intermediate-Advanced — 3-Process Style

5.1 Why a Third Process?

The 2-process style mixes next-state logic and output logic in one process. The **3-process** style separates them for clarity and avoids accidental latch inference in large FSMs.

Process	Sensitive to	Responsibility
1	`reset, clk`	State register (flip-flops only)
2	`state, inputs`	Next-state logic only
3	`state` (Moore) or `state, inputs` (Mealy)	Output logic only

5.2 Problem — Traffic Light Controller

A simple 3-state traffic light: **Green** → **Yellow** → **Red** → **Green** ... Input `timer` goes high when the current phase times out. Outputs: `green`, `yellow`, `red`.

Level 4 — traffic_light.vhd

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity traffic_light is
    port (
        clk    : in  std_logic;
        reset  : in  std_logic;
        timer  : in  std_logic;   -- '1' when phase timer expires
        green  : out std_logic;
        yellow : out std_logic;
        red    : out std_logic
    );
end traffic_light;

architecture myfsm of traffic_light is
    -- state encoding
    type state_type is (GREEN, YELLOW, RED);

    -- signals
    signal state, nextState : state_type;

begin

    -- ----------------------------------------
    -- Process 1 : State Register
    -- ----------------------------------------
```

```vhdl
    process (reset, clk)
    begin
        if reset = '1' then
            state <= GREEN;
        elsif rising_edge(clk) then
            state <= nextState;
        end if;
    end process;

    -- ----------------------------------------
    -- Process 2 : Next-State Logic ONLY
    -- ----------------------------------------
    process (state, timer)
    begin
        nextState <= state;   -- default: hold state

        case state is

            when GREEN =>
                if timer = '1' then
                    nextState <= YELLOW;
                end if;

            when YELLOW =>
                if timer = '1' then
                    nextState <= RED;
                end if;

            when RED =>
                if timer = '1' then
                    nextState <= GREEN;
                end if;

            when others =>
                nextState <= GREEN;

        end case;
    end process;

    -- ----------------------------------------
    -- Process 3 : Output Logic ONLY (Moore)
    -- Sensitive only to state (pure Moore)
    -- ----------------------------------------
    process (state)
    begin
        -- default all outputs off
        green  <= '0';
        yellow <= '0';
        red    <= '0';

```

```
        case state is
            when GREEN  => green  <= '1';
            when YELLOW => yellow <= '1';
            when RED    => red    <= '1';
            when others => null;
        end case;
    end process;

end myfsm;
```

3-Process Advantages

- Output process is **pure combinational** and easy to read.
- Changing output logic never accidentally touches next-state logic.
- Synthesis tools can optimize each process independently.
- Easier to convert between Moore and Mealy (just change Process 3).

6 Level 5 — State Encoding Strategies

❍ Level 5 · Advanced — Binary, Gray Code, One-Hot

6.1 Three Encoding Methods

VHDL lets you declare states symbolically and let the tool choose encoding, or you can **force** a specific encoding using constants.

Method	Bits needed	Flip-flops	Best for
Binary	$\lceil \log_2 N \rceil$	Fewest	Low resource designs
Gray	$\lceil \log_2 N \rceil$	Fewest	Low EMI / glitch sensitive
One-Hot	N	Most	Fast FPGAs (LUT efficiency)

6.2 Forcing Encoding in VHDL

Level 5 — state_encoding.vhd

```vhdl
library ieee;
use ieee.std_logic_1164.all;

-- ---------------------------------------------
-- METHOD A : Binary Encoding (manual)
-- 3 states need 2 bits: S0=00, S1=01, S2=10
-- ---------------------------------------------
architecture binary_enc of fsm_enc is

    constant S0 : std_logic_vector(1 downto 0) := "00";
    constant S1 : std_logic_vector(1 downto 0) := "01";
    constant S2 : std_logic_vector(1 downto 0) := "10";

    signal state, nextState : std_logic_vector(1 downto 0);

begin
    -- processes same as before, just use std_logic_vector
    process (reset, clk)
    begin
        if reset = '1' then
            state <= S0;
        elsif rising_edge(clk) then
            state <= nextState;
        end if;
    end process;

    process (state, X)
    begin
        nextState <= S0;
        Z         <= '0';
```

```

        case state is
            when S0 =>
                if X = '1' then nextState <= S1; end if;
            when S1 =>
                if X = '1' then nextState <= S2;
                else            nextState <= S0; end if;
            when S2 =>
                Z <= '1';
                nextState <= S0;
            when others =>
                nextState <= S0;
        end case;
    end process;

end binary_enc;


-- ---------------------------------------------
-- METHOD B : One-Hot Encoding
-- 3 states need 3 bits: S0=001, S1=010, S2=100
-- ---------------------------------------------
architecture onehot_enc of fsm_enc is

    constant S0 : std_logic_vector(2 downto 0) := "001";
    constant S1 : std_logic_vector(2 downto 0) := "010";
    constant S2 : std_logic_vector(2 downto 0) := "100";

    signal state, nextState : std_logic_vector(2 downto 0);

begin
    process (reset, clk)
    begin
        if reset = '1' then
            state <= S0;        -- only bit 0 set
        elsif rising_edge(clk) then
            state <= nextState;
        end if;
    end process;

    process (state, X)
    begin
        nextState <= S0;
        Z         <= '0';

        case state is
            when S0 =>
                if X = '1' then nextState <= S1; end if;
            when S1 =>
                if X = '1' then nextState <= S2;
```

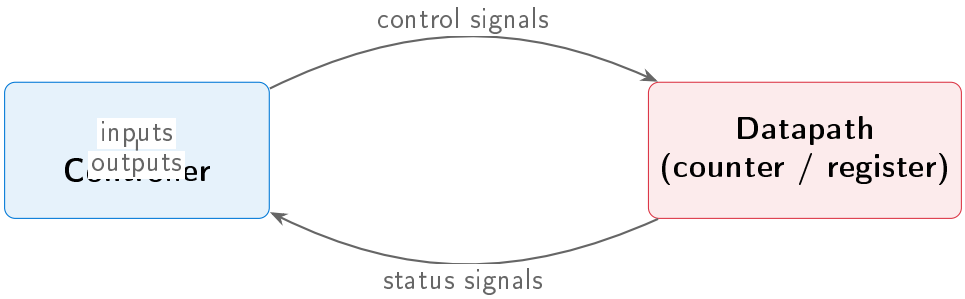
```
                    else                nextState <= S0; end if;
                when S2 =>
                    Z <= '1';
                    nextState <= S0;
                when others =>
                    nextState <= S0;  -- fault recovery
            end case;
        end process;

end onehot_enc;


-- ----------------------------------------------
-- METHOD C : Let Quartus decide (RECOMMENDED)
-- Add synthesis attribute in your QSF file:
--   set_global_assignment -name STATE_MACHINE_PROCESSING ONE-HOT
-- Or use the Quartus GUI: Assignments > Settings >
--   Compiler Settings > Advanced Settings (Synthesis)
-- ----------------------------------------------
architecture auto_enc of fsm_enc is

    -- Just use enumerated type; Quartus picks best encoding
    type state_type is (S0, S1, S2);
    signal state, nextState : state_type;

begin
    -- ... same 2-process template ...
end auto_enc;
```

7 Level 6 — FSM with Datapath (FSMD)

★ Level 6 · Advanced — FSM + Counter/Register Datapath

7.1 What is an FSMD?

In real designs, an FSM alone is not enough. You combine an FSM (**controller**) with a **datapath** (counters, registers, adders) to build complex digital systems.



7.2 Problem — Pulse-Width Counter

Count how many clock cycles input `X` stays high. When `X` goes low, output the 4-bit count `CNT` and assert `done = 1` for one cycle.

Level 6 — pulse_counter_fsmd.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;          -- needed for unsigned counter

entity pulse_counter_fsmd is
    port (
        clk   : in  std_logic;
        reset : in  std_logic;
        X     : in  std_logic;
        CNT   : out std_logic_vector(3 downto 0);
        done  : out std_logic
    );
end pulse_counter_fsmd;

architecture myfsm of pulse_counter_fsmd is

    -- -- FSM (controller) -----------------------
    type state_type is (IDLE, COUNTING, REPORT);
    signal state, nextState : state_type;

    -- -- Datapath signals -----------------------
    signal count      : unsigned(3 downto 0);
```

```vhdl
    signal count_en  : std_logic;   -- control: enable counter
    signal count_clr : std_logic;   -- control: clear counter

begin

    -- ----------------------------------------
    -- Datapath : 4-bit counter
    -- ----------------------------------------
    process (reset, clk)
    begin
        if reset = '1' then
            count <= (others => '0');
        elsif rising_edge(clk) then
            if count_clr = '1' then
                count <= (others => '0');
            elsif count_en = '1' then
                count <= count + 1;
            end if;
        end if;
    end process;

    -- ----------------------------------------
    -- Process 1 : State Register
    -- ----------------------------------------
    process (reset, clk)
    begin
        if reset = '1' then
            state <= IDLE;
        elsif rising_edge(clk) then
            state <= nextState;
        end if;
    end process;

    -- ----------------------------------------
    -- Process 2 : Next-State Logic
    -- ----------------------------------------
    process (state, X)
    begin
        nextState <= state;

        case state is
            when IDLE =>
                if X = '1' then
                    nextState <= COUNTING;
                end if;

            when COUNTING =>
                if X = '0' then
```

```
                    nextState <= REPORT;
                end if;

            when REPORT =>
                nextState <= IDLE;    -- stay 1 cycle then go idle

            when others =>
                nextState <= IDLE;
        end case;
    end process;

    -- ----------------------------------------
    -- Process 3 : Output + Control Signals
    -- ----------------------------------------
    process (state, count)
    begin
        -- defaults
        count_en  <= '0';
        count_clr <= '0';
        done      <= '0';
        CNT       <= (others => '0');

        case state is
            when IDLE =>
                count_clr <= '1';     -- keep counter cleared

            when COUNTING =>
                count_en  <= '1';     -- increment every cycle X is high

            when REPORT =>
                done <= '1';
                CNT  <= std_logic_vector(count);

            when others =>
                null;
        end case;
    end process;

end myfsm;
```

8 Common Mistakes & How to Fix Them

These bugs were found in a real student file!

The following table documents the **exact errors** from a typical first attempt. Study each one carefully.

8.1 Bug Reference Table

Location	✗ Wrong	✓ Correct
Port direction of reset, clk, X	`reset : out`	`reset : in`
Port direction of reset, clk, X	`X, clk : out`	`X, clk : in`
Signal declaration	`signal state, nextState, stateType ;`	`signal state, nextState : state_type;`
State register if-condition	`if reset = '1'` (missing then)	`if reset = '1' then`
Clock edge function	`riding_edge(clk)`	`rising_edge(clk)`
End of if statement	`end;`	`end if;`
Duplicate case branch	`when S0 => ...` `when S0 => ...`	Each state appears exactly ONCE
Missing output state	`when S2 missing ...`	Every declared state must have a when
Missing catch-all	(no when others)	`when others => nextState <= S0;`
Architecture end	`end myfsm`	`end myfsm;`

8.2 The Corrected Version of Your fsm.vhd

fsm.vhd — Corrected

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity fsm is
    port (
        clk   : in  std_logic;   -- FIX: was "out"
        reset : in  std_logic;   -- FIX: was "out"
        X     : in  std_logic;   -- FIX: was "out"
        Z     : out std_logic
    );
end fsm;

architecture myfsm of fsm is
    -- state encoding
    type state_type is (S0, S1, S2, S3, S4, S5, S6);

    -- FIX: missing ": state_type" and removed stateType
    signal state, nextState : state_type;

begin

    -- ----------------------------------------
    -- Process 1 : State Register
    -- ----------------------------------------
    process (reset, clk)
    begin
        if reset = '1' then        -- FIX: added "then"
            state <= S0;
        elsif rising_edge(clk) then    -- FIX: "riding" -> "rising"
            state <= nextState;
        end if;                    -- FIX: "end;" -> "end if;"
    end process;

    -- ----------------------------------------
    -- Process 2 : Next-State + Output Logic
    -- ----------------------------------------
    process (state, X)
    begin
        -- defaults prevent latches
        nextState <= S0;
        Z         <= '0';

        case state is

            -- S0
```

```
            when S0 =>
                if X = '0' then
                    nextState <= S1;
                    Z         <= '1';
                else
                    nextState <= S0;
                    Z         <= '0';
                end if;

            -- S1
            when S1 =>
                if X = '0' then
                    nextState <= S3;
                    Z         <= '1';
                else
                    nextState <= S4;
                    Z         <= '0';
                end if;

            -- S2
            when S2 =>
                if X = '0' then
                    nextState <= S4;
                    Z         <= '0';
                else
                    nextState <= S4;
                    Z         <= '1';
                end if;

            -- FIX: S3, S4, S5, S6 all need their own when clause
            when S3 =>
                nextState <= S0;
                Z         <= '0';

            when S4 =>
                nextState <= S0;
                Z         <= '0';

            when S5 =>
                nextState <= S0;
                Z         <= '0';

            when S6 =>
                nextState <= S0;
                Z         <= '0';

            -- FIX: always include when others
            when others =>
                nextState <= S0;
```

```
                    Z          <= '0';

            end case;
        end process;

end myfsm;    -- FIX: added semicolon
```

9 Quick Reference — The Master Template

Master 2-Process FSM Template (Copy & Adapt)

```
library ieee;
use ieee.std_logic_1164.all;

entity YOUR_FSM_NAME is
    port (
        clk   : in  std_logic;
        reset : in  std_logic;
        -- add your inputs here  (always "in")
        -- add your outputs here (always "out")
    );
end YOUR_FSM_NAME;

architecture myfsm of YOUR_FSM_NAME is
    type state_type is (S0, S1, S2 /*, add more states */);
    signal state, nextState : state_type;

begin

    -- ------------------------------------------
    -- Process 1 : State Register  [NEVER changes]
    -- ------------------------------------------
    process (reset, clk)
    begin
        if reset = '1' then
            state <= S0;
        elsif rising_edge(clk) then
            state <= nextState;
        end if;
    end process;

    -- ------------------------------------------
    -- Process 2 : Next-State + Output  [All logic here]
    -- ------------------------------------------
    process (state /*, add all inputs to sensitivity list
        */)
    begin
        -- ALWAYS write defaults first
        nextState <= S0;
        -- YOUR_OUTPUT <= '0';

        case state is

            when S0 =>
                -- YOUR_OUTPUT <= ...;
                if (/* condition */) then
                    nextState <= S1;
```

```
                    else
                        nextState <= S0;
                    end if;

                when S1 =>
                    -- add more states ...
                    nextState <= S0;

                -- ALWAYS include this
                when others =>
                    nextState <= S0;

            end case;
        end process;

end myfsm;
```

9.1 FSM Design Checklist

- ✓ All ports: `clk`, `reset`, inputs are `in`; outputs are `out`.
- ✓ Signal declaration: `signal state, nextState : state_type;` (type specified!).
- ✓ State register uses `rising_edge(clk)` (not `riding_edge`).
- ✓ State register if-clause ends with `end if;` (not `end;`).
- ✓ Every state in `state_type` has exactly **one** `when` clause.
- ✓ `when others` is the last clause in every `case`.
- ✓ Default assignments written at the **top** of Process 2.
- ✓ All inputs added to the sensitivity list of Process 2.
- ✓ Architecture ends with `end myfsm;` (semicolon!).
- ✓ No duplicate `when` labels in any `case` block.
- ✓ Outputs never driven from Process 1 (state register).

9.2 Level Summary Table

Level	Topic	States	Processes	New Skill
1	Toggle (Moore)	2	2	Basic template, port directions
2	Sequence Detector Moore	5	2	Multi-state, overlapping detection
3	Sequence Detector Mealy	4	2	Output on transitions, fewer states
4	Traffic Light (3-proc)	3	3	Separated output process, multi-output
5	State Encoding	Any	2	Binary, Gray, One-Hot, auto
6	FSMD Pulse Counter	3	3+	FSM + datapath, control signals

9.3 The Golden Rules (Never Break These)

1. **Never** drive the same signal from two different processes.
2. **Never** put `rising_edge` in a combinational process (Process 2 or 3).
3. **Always** write default assignments before `case` in any combinational process — avoids inferred latches.
4. **Always** include `when others` — an FPGA can power up in an unknown state.
5. **Never** read an output port inside the architecture — declare an internal signal, drive the output from it.